

16. JSON en Python

ARCHIVOS `.JSON`, `DUMP()`, `LOAD()` Y PRACTICA INCREMENTAL

Programación I
Ingeniería Electrónica y Telecomunicaciones

Comisión 3
Profesor: Lautaro Linquimán
Universidad Nacional de Río Negro



Repaso express

Clase anterior

1. ¿Para qué nos servía pasar datos por `sys.argv` en vez de escribirlos dentro del programa?
2. Cuando un programa recibe una ruta de archivo, ¿qué conviene validar antes de abrirlo?
3. Si un dato llega desde afuera del programa, ¿por qué no conviene asumir que siempre viene bien?

Archivos JSON

JSON (Javascript Object Notation) es un formato de texto usado para **guardar** y **compartir** datos estructurados dado a que es un estandar.

Suele aparecer cuando:

- un programa guarda informacion en un archivo;
- una app manda datos a otra;
- queremos dejar datos legibles para personas y programas.

No lo vemos, pero la mayoria de los sitios web maneja JSON para su comunicación con el navegador.

Esto es un archivo JSON

```
{  
  "local": "Kwik-E-Mart",  
  "abierto": true,  
  "caja_actual": null,  
  "productos": [  
    {  
      "nombre": "Rosquillas", "precio": 1200, "stock": 14  
    },  
    {  
      "nombre": "Coca", "precio": 950, "stock": 9  
    }  
  ]  
}
```

Más info: <https://es.wikipedia.org/wiki/JSON>

Formato JSON

- Usa llaves `{ }` para objetos.
- Usa corchetes `[ ]` para arrays.
- Las claves van entre comillas dobles.
- Aparecen `true`, `false` y `null`.
- Las comas separan elementos, pero no puede sobrar la ultima.

Estructura de datos: Python y JSON

Python	JSON
<code>dict</code>	<code>object</code>
<code>list</code>	<code>array</code>
<code>None</code>	<code>null</code>
<code>True</code> / <code>False</code>	<code>true</code> / <code>false</code>

- `tuple` Python json la convierte en un array JSON; al cargarla vuelve como list.
- `set` no se puede serializar directamente y produce `TypeError`.

Nota: No se puede convertir en JSON cualquier objeto python, tienen que ser objetos serializables y compatibles con JSON.

JSON valido vs JSON invalido

Valido

```
{  
  "nombre": "Lisa",  
  "activo": true  
}
```

Invalido

```
{  
  nombre: 'Lisa',  
  "activo": true,  
}
```

Errores tipicos:

- clave sin comillas;
- comillas simples;
- coma sobrante al final.

Se pueden usar el ide o <https://jsonchecker.com/> para verificar la validez de nuestro JSON.

Escribir JSON desde Python

```
import json

producto = {
    "nombre": "Coca",
    "precio": 950,
    "stock": 9
}

with open("producto.json", "w") as archivo:
    json.dump(producto, archivo)
```

`json.dump()` recibe una estructura Python y la escribe como JSON.

Emprolijando el JSON con **indent**

```
import json

datos = {
    "local": "Kwik-E-Mart",
    "abierto": True,
    "productos": ["Rosquillas", "Coca"]
}

with open("local.json", "w") as archivo:
    json.dump(datos, archivo, indent=2)
```

Con **indent**, el archivo queda mucho mas legible.

Volver a cargar datos

```
import json

with open("producto.json", "r") as archivo:
    datos = json.load(archivo)

print(datos)
print(type(datos))
```

`json.load()` lee el archivo JSON y devuelve estructuras de Python.

Leer, recorrer y modificar

```
import json

with open("local.json", "r") as archivo:
    datos = json.load(archivo)

for producto in datos["productos"]:
    print(producto)

datos["abierto"] = False
datos["caja_actual"] = "caja_2"
```

Después de `load()`, volvemos a tener listas, diccionarios y valores de Python.

Ejercicio corto 1

Usando `guardia_house.json`:

1. Cargar el archivo con `json.load()`.
2. Mostrar el nombre de la guardia.
3. Recorrer la lista de pacientes.
4. Mostrar solo los pacientes con prioridad `"alta"`.

Ejercicio integrador de clase

Vamos a convertir el siguiente archivo en un JSON.

A partir del siguiente archivo [sensores_transito_caba.txt](#).

Formato del archivo

```
1023;Avenida Cabildo;Juramento;Belgrano;13;2019  
1058;Avenida Rivadavia;Medrano;Almagro;5;2021  
...
```

Cada linea tiene:

```
id_sensor;calle;cruce;barrio;comuna;anio_instalacion
```

Ejercicio integrador de clase

Con ese archivo:

1. Leer el archivo, separar campos, limpiar y validar lo minimo:
 - que haya 6 campos;
 - que `id_sensor`, `comuna` y `anio_instalacion` sean numericos;
 - que calle, cruce y barrio no queden vacios.

Los que se animen, lean la ruta al archivo mediante `sys.argv`

2. Armar una lista de sensores validos y guardarla en `sensores_caba.json`.
3. Volver a cargar ese JSON y hacer una consulta o una modificacion simple.

Opciones para seguir

Con el JSON ya cargado:

1. Mostrar los sensores de una comuna elegida.
2. Contar cuántos sensores validos quedaron.
3. Explicar qué linea quedó afuera y por qué.

FIN

Veamos un poco del TP integrador